

Projekt 1: *Regresja i klasyfikacja*

Paweł Bielecki, Kamil Błazejczyk

Raport z projektu nr 1 na przedmiocie *Wprowadzenie do sztucznej inteligencji* na semestrze letnim 2026L na wydziale Matematyki i Nauk Informatycznych Politechniki Warszawskiej.

Spis treści

1. Informacja na temat wykorzystania AI	3
2. Regresja	4
2.1. Opis zadania	4
2.2. Opis zbiorów danych	4
2.2.1. Preprocessing danych	4
2.2.1.1. Kolumna Id	4
2.2.1.2. Outliery	4
2.2.1.3. Brakujące dane	5
2.2.1.4. Kolumny opisowe	5
2.2.1.5. Kolumny jakości	5
2.2.1.6. Uzupełnianie brakujących danych	5
2.3. Opis eksperymentów	5
2.4. Opis wyników	5
2.5. Konkluzje i propozycje dalszych badań	5
2.6. Podział pracy między członkami grupy	5

1. Informacja na temat wykorzystania AI

Przy pracy nad raportem AI służyło nam przede wszystkim pomocą w redagowaniu tekstu. Całe zadanie staraliśmy się wykonać sami (z pomocą dokumentacji `pandas` i `sklearn`). Fragmenty raportu wygenerowane przez AI lub napisane z jego pomocą są ujęte w raporcie w cudzysłowy francuskie: » «.

2. Regresja

2.1. Opis zadania

Zadanie polegało na przeprowadzeniu analizy regresji w celu przewidzenia cen nieruchomości oraz na porównaniu skuteczności różnych algorytmów uczenia maszynowego.

2.2. Opis zbiorów danych

» Podstawą naszych działań był zbiór danych *Ames Housing*, stanowiący użyteczną alternatywę dla klasycznego zbioru *Boston Housing Data Set*. Dane te pochodzą z biura rzeczoznawcy w mieście Ames w stanie Iowa i obejmują informacje o indywidualnych nieruchomościach mieszkalnych, które zostały sprzedane w latach 2006–2010. Zbiór ten został pierwotnie przygotowany na potrzeby końcowosemestralnego projektu dla studentów kursu z analizy regresji. Zawiera on informacje bardzo zbliżone do tych, jakich typowy nabywca domu szukałby przed podjęciem decyzji o zakupie, co czyni go idealnym materiałem do przewidywania cen.

Sam zbiór jest niezwykle szczegółowy i obszerny – składa się z 1460 obserwacji oraz kilkunastu zmiennych. Dokładna struktura to 82 kolumny, w tym 2 zmienne identyfikacyjne oraz 23 zmienne nominalne, 23 porządkowe, 14 dyskretnych i 20 ciągłych. Zmienną docelową (zależną), którą algorytmy mają za zadanie przewidzieć, jest zmienna ciągła o nazwie `SalePrice`, reprezentująca ostateczną cenę sprzedaży nieruchomości w dolarach. Cechy opisujące poszczególne domy obejmują niemal każdy możliwy parametr: od fizycznej lokalizacji w konkretnej dzielnicy, przez ogólną ocenę jakości i stanu technicznego materiałów, aż po detale takie jak powierzchnia działki, rodzaj ogrzewania, udogodnienia w piwnicy czy pojemność garażu.

Kluczowym aspektem z punktu widzenia budowy modeli, o którym wyraźnie wspomina dokumentacja, jest odpowiednia obsługa wartości odstających. W zbiorze znajduje się 5 nietypowych obserwacji, które można łatwo zauważyć na wykresie zależności ceny sprzedaży od powierzchni mieszkalnej. Trzy z nich to tzw. sprzedaże częściowe (które prawdopodobnie nie odzwierciedlają rzeczywistych wartości rynkowych), a dwie to po prostu bardzo duże domy. Aby zapewnić lepsze działanie algorytmów i stworzyć model przewidujący typowe ceny rynkowe, autor dokumentacji zaleca usunięcie przed analizą wszystkich domów o powierzchni przekraczającej 4000 stóp kwadratowych, co skutecznie eliminuje te anomalie. «

2.2.1. Preprocessing danych

2.2.1.1. Kolumna `Id`

Kolumna `Id` z pewnością nie jest skorelowana z ceną mieszkania, zatem nie powinna być brana pod uwagę:

```
data = data.drop("Id", axis="columns")
```

2.2.1.2. Outliery

Jest kilka domów o wielkiej powierzchni, sprzedanych po normalnej cenie. Obserwacje te zostały usunięte z dalszej analizy:

```
data = data[data["GrLivArea"] < 4_000]
```

2.2.1.3. Brakujące dane

Brakujące dane są reprezentowane jako '?'. Zamieniamy je na `pd.NA` aby potem dokonać interpolacji (opisane później):

```
data[data == '?'] = pd.NA
```

2.2.1.4. Kolumny opisowe

Dla każdego wariantu danej opisowej dodajemy kolumnę z wartością 0 lub 1 w zależności od tego, czy dany wiersz posiadał tą daną:

```
data = pd.get_dummies(data, dtype=int)
```

2.2.1.5. Kolumny jakości

Zbiór danych posiada kilka kolumn opisujących jakość (`Ex` - excellent, `Gd` - good itd.). Można zostawić je jako typ `str` ale to powoduje stworzenie dużej liczby kolumn (przez `pd.get_dummies`). Aby tego uniknąć zamieniliśmy je na odpowiednie liczby:

```
def qual_to_val(column):
    qual_to_val = {"Ex": "6", "Gd": "5", "TA": "4", "Fa": "3", "Po": "2", "NA":
"0", pd.NA: "0"}
    data[column] = data[column].map(qual_to_val)
    data[column] = pd.to_numeric(data[column])
    data[column] = data[column].interpolate()

for col in ["ExterQual", "ExterCond", "BsmtQual", "BsmtCond", "HeatingQC",
"KitchenQual", "FireplaceQu", "GarageQual", "GarageCond", "PoolQC"]:
    qual_to_val(col)
```

2.2.1.6. Uzupełnianie brakujących danych

Brakujące dane zostały zinterpolowane (liniowo), można również użyć `ffill` i `bfill` ale zauważyliśmy, że nie robi to praktycznej różnicy:

```
data[column] = data[column].interpolate()
```

2.3. Opis eksperymentów

2.4. Opis wyników

2.5. Konkluzje i propozycje dalszych badań

2.6. Podział pracy między członkami grupy